

<u>University No.</u>	<u>Name</u>	<u>Checkoff 1</u>
<u>University No.</u>	<u>Name</u>	<u>Checkoff 2</u>
<u>University No.</u>	<u>Name</u>	<u>Checkoff 3</u>

Objective: Exploiting Python for calculating probability

1 Overview

In previous labs you experimented using Python to enumerate all possible choices in a given problem. In this lab, you will build upon this skill to further deploy Python for calculating probability of different events happening and related expectation values. This lab will also offer further practice using functions, if-statements, loops and dictionaries in Python.

2 Getting the Code

Download the corresponding files for this lab from Moodle.

3 The Birthday Problem

Consider a room with N people. What is the probability that at least two of them share the same birthday? This seems to happen unexpectedly often!

3.1 Coding: Calculating the Probability Open the file `birthday.py`. The file has a skeleton of a Python program that uses random numbers to calculate the probability of at least two people sharing the same birthday, when there are N people around. The program has some similarity to the one shown in class for calculating the value of π . Complete the program by filling in the missing code in the place marked with the comment `YOUR CODE HERE`.

If the code you add runs smoothly, the program should also be able to produce a graph showing “the probability of sharing birthday” versus “the number of people” around. See if the graph makes sense to you.

Note that you may need to close that graph in order to re-run the Python program smoothly without any error messages.

3.2 Checkoff 1

- Show your working code in `birthday.py` to your TA. Explain what you have done.
- What is the probability of two people sharing the same birthday when there are 40 people around? If we want this probability to be 50%, how many people should be around?
- The code in `birthday.py` uses iterations to calculate the probability. Explore the minimum number of iterations, below which the calculated probability becomes untrustworthy.

4 Detecting AI Plagiarism: We Gotta Be Careful with Probabilities

We often need to use probabilities to describe what happens around us. But making deductions based on probabilities can be tricky, as we need to be careful about what we know and which probabilities we are talking about. Here we will explore this in the context of detecting plagiarism from artificial intelligence (AI).

4.1 The Context The College of Pythonic Sorcery (CPS) is adopting an AI detector to scan each homework submitted by their students, with the aim to identify homework produced by AI instead of the students. Data from the previous years show that $p = 1\%$ of students' homeworks were indeed substantially created by AI.

This AI detector is quite accurate but still imperfect: when scanning a homework produced by AI, the detector may fail to detect it 5% of the time. On the other hand, a homework done by a human student may falsely be misjudged as an AI creation with a probability of 3%.

William submitted his homework and unfortunately received a notification saying that his homework was flagged as AI plagiarism. How confident are we that William had indeed committed AI plagiarism? We will quantify this "confidence" through simulating what may happen among 100,000 student homeworks, i.e., to calculate the conditional probability of a student cheating with AI given that the AI detector had flagged about this. We denote this probability as $P(\text{cheating}|\text{flagged})$.

4.2 Coding: Simulating 100,000 Homeworks Open the file `AIdetector.py`. In this program, we would like to generate a list of 100,000 elements. Each element represent one homework. The numerical value of an element carries different meanings:

- -2 = the homework is an honest submission but was flagged as plagiarism;
- -1 = the homework was written by AI and was correctly flagged as plagiarism;
- 1 = the homework was written by AI but not flagged as plagiarism;
- 2 = the homework is an honest submission and was not flagged as plagiarism.

Complete the code in `AIdetector.py`, e.g., through filling in the parts marked by the comment `YOUR CODE HERE`. The idea is to generate a list of 100,000 elements to represent 100,000 homework and then use this list to calculate $P(\text{cheating}|\text{flagged})$.

4.3 Checkoff 2

- Show your working code in `AIdetector.py` to your TA. Explain what you have done.
- What is the probability of William cheating with AI, given that his homework has been flagged? What kind of judgment would you make for William's case?
- What happens when p , the percentage of true cheating homework, changes?

5 Knapsack Comes Back to Haunt Us with Items Having Uncertain Values

The world around us is full of uncertainties! Here let's revisit our old friend, the Knapsack Problem. But this time the items have uncertain values (i.e., the value of an item follows a particular probability distribution), while the weight of an item is still fixed (thank god!).

5.1 Coding: Greedy Algorithm by Focusing on the Smallest / Largest Possible Value of an Item

Open the file `knapsack_VariableValues.py`. While some parts marked with the comment `YOUR CODE HERE` need your completion, the Python program in this file should look familiar: the main body is what we have used in Lab 1 before! We are still selecting items into our knapsack, subject to the constraint that the total `weight` ≤ 15 .

This time the item list is “slightly” different, compared to the one in Lab 1. The obvious change is that its name has been modified to `probabilistic_items` to reflect the difference. But it is still a list of items in which every item is represented as a tuple in the form

```
(name, value, weight)
```

But now the `value` of an item has been extended into a dictionary to represent its probability distribution in the form

```
{value1: probability1, value2: probability2, ...}
```

If `my_dict` is the name of this dictionary, then the Python expression `my_dict[value1]` will return `probability1` to us.

For example, for the item ‘chocolate’, we see that its `value` is represented by

```
{70: 0.50, 110: 0.30, 160: 0.20}
```

This means 50% of the time the value of the chocolate is 70, 30% of the time its value is 110, and its value goes to 160 for 20% of the time.

Remember in Lab 1, we sort our item list based on a key (and sorting with different key may lead to different items being selected by a greedy algorithm). Now let's sort based on the lowest possible value of an item. Apply a greedy algorithm to see which items are to be selected. Hint: the Python command `next(iter())` will be your friend to accomplish this task.

Next let's sort based on the highest possible value of an item. Apply a greedy algorithm to see which items are to be selected. Hint: the Python command `next(reversed())` will be your good friend this time.

5.2 Finding the Expected Value The expected value can be a good measure to gauge the “effect” of a quantity with uncertain values. In the file `knapsack_VariableValues.py`, complete the function that calculates the expected value of `value` for an item (which itself is a tuple with the probabilistic `value` stored as the second element). For instance,

- `probabilistic_items[6]` stores the information about the chocolate; and
- `probabilistic_items[6][1]` is a dictionary capturing the possible `value` of chocolate under different probabilities.

Hint: if

```
x = ('chocolate', {70: 0.50, 110: 0.30, 160: 0.20}, 1)
```

then

- `x[1][70]` represents 0.50, and similarly for the other **value** and **probability**;
- `x[1].keys()` is a useful command that returns an iterable containing all the possible **values** the chocolate can have.

Sort the item list based on the expected value of an item. Apply a greedy algorithm, and see which items will be selected.

5.3 Oh, the Knapsack is actually my Investment Portfolio?! The Knapsack Problem in this part is actually not a contrived example to force you to apply the concept of “expected values”. Now imagine each “item” represents a certain financial product (bonds, stocks, options, ETFs, accumulators, etc.), and so this “knapsack” is actually an investment portfolio. The weight of an item denotes how much it burdens you to invest in this product (and hence there is an upper bound on how much “**weight**” you can sustain unless your surname is Musk). The probabilistic nature of an item’s **value** then captures the uncertain value of a product in the financial market.

Reflect on the findings of your greedy algorithms. Realising you are actually dealing with an investment portfolio, would you make the same recommendation to your grandma or your brother?

5.4 Checkoff 3

- Show your working code in `knapsack.VariableValues.py` to your TA. Explain what you have done (e.g., how you have defined the function `expected_value(x)`) and results of your greedy algorithms.
- Explain to your TA whether you will recommend the findings of your greedy algorithms to your grandma or your brother, if you are actually analysing an investment portfolio.